

Sistemas Operacionais

Um pouco de história

Professor Romário da Silva Borges

O que veremos nesta aula

01

Primeiras máquinas de computação

Charles Babbage e Ada Lovelace

02

Primeira geração

Válvulas e painéis de conectores

03

Segunda geração

Transistores e sistemas em lote

04

Terceira geração

Circuitos integrados e multiprogramação

05

Quarta geração

Computadores pessoais e SOs modernos

06

Legado

Técnicas que permanecem até hoje

Se hoje conseguimos abrir vários programas ao mesmo tempo, imprimir documentos, navegar na internet e salvar arquivos sem entender o hardware... em que momento da história surgiu a necessidade de um sistema operacional?

Por que olhar para o passado?

SOs não surgiram prontos

Cada geração resolveu problemas específicos de sua época, de forma incremental.

Conceitos antigos, uso atual

Muitos mecanismos modernos têm raízes em sistemas criados décadas atrás.

Hardware guia o software

A evolução do hardware criou novas exigências para o sistema operacional.

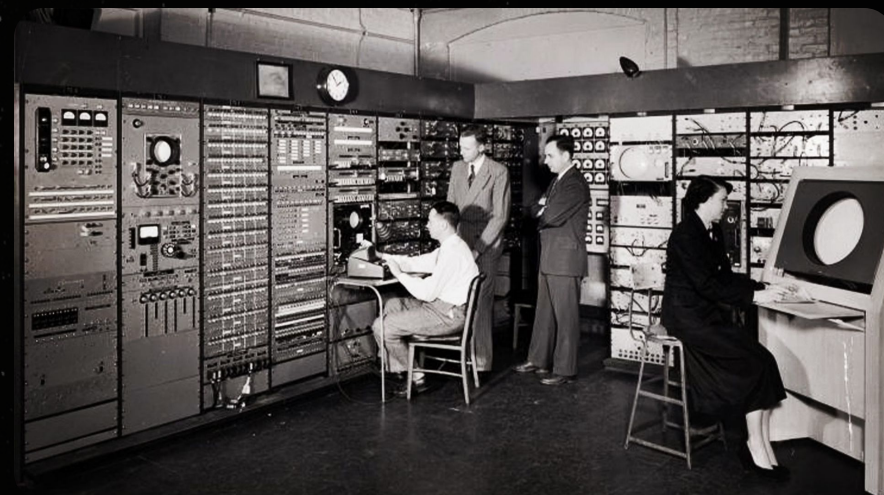
Compreender o porquê

Entender a história ajuda a explicar por que os SOs são como são hoje.

Antes do sistema operacional

A computação era feita de forma muito mais direta e manual:

- O usuário interagia quase diretamente com a máquina
- A programação dependia fortemente do hardware específico
- Tarefas diferentes exigiam reconfigurações manuais
- Não havia abstrações como arquivos, processos ou permissões
- O sistema operacional surge justamente para **reduzir essa complexidade**.





Século XIX

Charles Babbage e a automação dos cálculos

Em meados do século XIX, havia grande demanda por cálculos precisos em navegação, engenharia e pesquisa científica. Babbage idealizou **máquinas capazes de realizar cálculos automaticamente**, especialmente para construção de tabelas matemáticas — eliminando os erros humanos que eram frequentes na época.

1834 - 1871

A máquina diferencial

Projetada para resolver cálculos baseados em **diferenças entre números**, permitindo gerar tabelas matemáticas com precisão e rapidez.

Já estavam presentes ideias essenciais que reconhecemos nos computadores modernos:

Entrada

Dados inseridos manualmente

Processamento

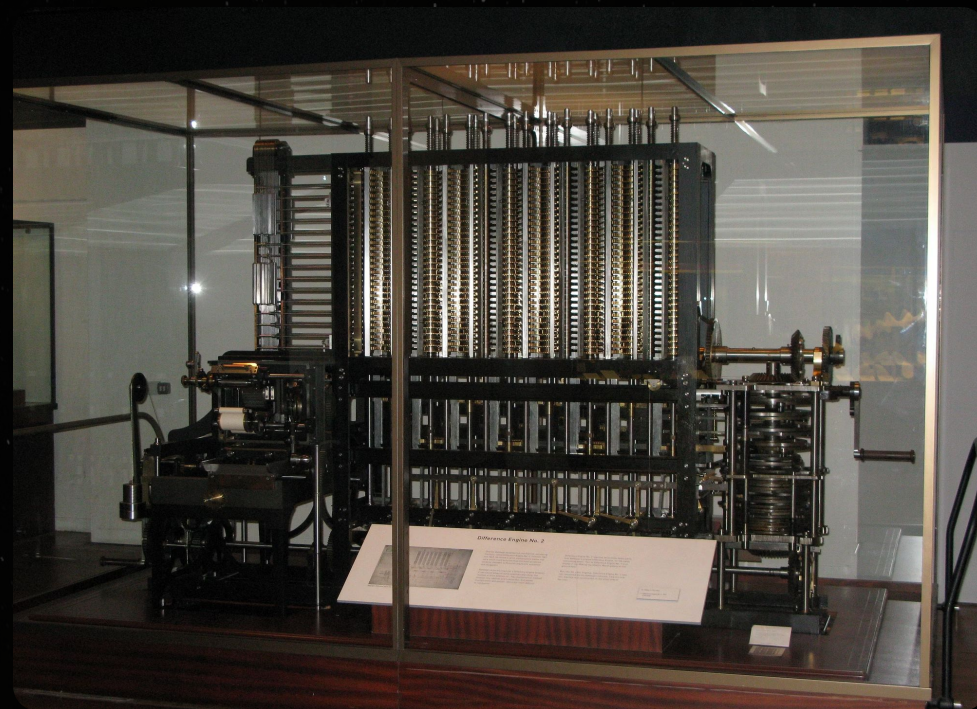
Cálculo automático

Armazenamento

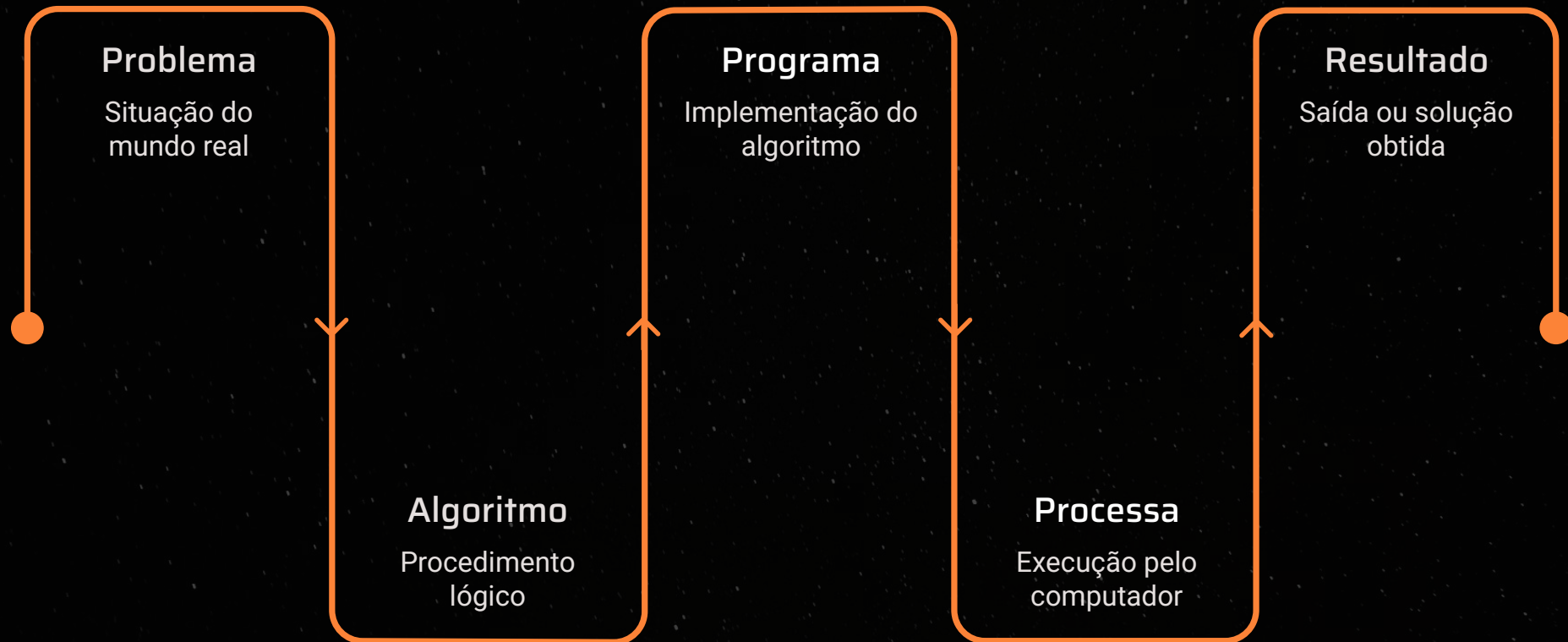
Resultados intermediários

Saída

Impressão de tabelas



Do problema ao resultado



Esse fluxo conceitual, presente na máquina diferencial de Babbage, **continua sendo o mesmo** nos computadores atuais. As tecnologias mudaram, mas a lógica fundamental permanece.

O que a máquina diferencial representou?



Computação automática

Uma das primeiras tentativas práticas de realizar cálculos sem intervenção humana constante.



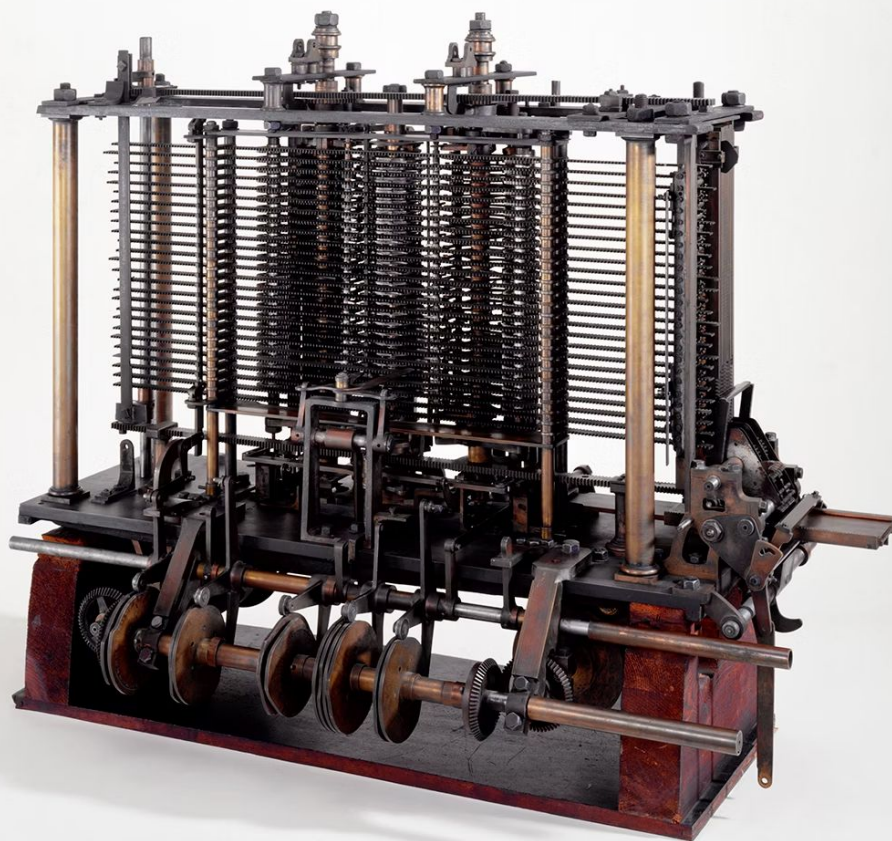
Inovação sob demanda

Nasceu de uma necessidade real da sociedade científica e industrial do século XIX.



Base conceitual

Fundou a ideia de máquinas programáveis que influenciou toda a computação posterior.



1837 - 1991

A Máquina analítica

Uma proposta muito mais ambiciosa:
uma **máquina de uso geral e programável**.

- Concebida para executar qualquer tipo de cálculo
- Possuía unidade aritmética, memória e controle
- Inspiração direta para o conceito moderno de computador
- Nunca foi plenamente construída — limitações técnicas, financeiras e políticas

Ada Lovelace

A ideia de programação

Babbage percebeu que uma máquina programável precisaria de **instruções organizadas**. Ada Lovelace é reconhecida historicamente como uma das primeiras pessoas a compreender o potencial real da programação — indo além dos cálculos, vislumbrando que a máquina poderia manipular qualquer símbolo, não apenas números. **É frequentemente chamada de primeira programadora da história.**

Number of Operation	Statement of Operation	Variables used upon	Variables resulting directly	Indication of change in the value on any Variable	Statement of Result	Index										Working Variable				Result Variable			
						γ_{11}	γ_{12}	γ_{13}	γ_{14}	γ_{15}	γ_{16}	γ_{17}	γ_{18}	γ_{19}	γ_{20}	γ_{21}	γ_{22}	γ_{23}	γ_{24}	γ_{25}	γ_{26}	γ_{27}	
						1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	
1	$\gamma_{11} = \gamma_{12}$	$\gamma_{11}, \gamma_{12}, \gamma_{13}$	$\gamma_{11} = \gamma_{12}$	$\gamma_{11} = \gamma_{12}$		2	n	2n	2n														
2	$\gamma_{12} = \gamma_{13}$	$\gamma_{12}, \gamma_{13}, \gamma_{14}$	$\gamma_{12} = \gamma_{13}$	$\gamma_{12} = \gamma_{13}$																			
3	$\gamma_{13} = \gamma_{14}$	$\gamma_{13}, \gamma_{14}, \gamma_{15}$	$\gamma_{13} = \gamma_{14}$	$\gamma_{13} = \gamma_{14}$																			
4	$\gamma_{14} = \gamma_{15}$	$\gamma_{14}, \gamma_{15}, \gamma_{16}$	$\gamma_{14} = \gamma_{15}$	$\gamma_{14} = \gamma_{15}$																			
5	$\gamma_{15} = \gamma_{16}$	$\gamma_{15}, \gamma_{16}, \gamma_{17}$	$\gamma_{15} = \gamma_{16}$	$\gamma_{15} = \gamma_{16}$																			
6	$\gamma_{16} = \gamma_{17}$	$\gamma_{16}, \gamma_{17}, \gamma_{18}$	$\gamma_{16} = \gamma_{17}$	$\gamma_{16} = \gamma_{17}$																			
7	$\gamma_{17} = \gamma_{18}$	$\gamma_{17}, \gamma_{18}, \gamma_{19}$	$\gamma_{17} = \gamma_{18}$	$\gamma_{17} = \gamma_{18}$																			
8	$\gamma_{18} = \gamma_{19}$	$\gamma_{18}, \gamma_{19}, \gamma_{20}$	$\gamma_{18} = \gamma_{19}$	$\gamma_{18} = \gamma_{19}$																			
9	$\gamma_{19} = \gamma_{20}$	$\gamma_{19}, \gamma_{20}, \gamma_{21}$	$\gamma_{19} = \gamma_{20}$	$\gamma_{19} = \gamma_{20}$																			
10	$\gamma_{20} = \gamma_{21}$	$\gamma_{20}, \gamma_{21}, \gamma_{22}$	$\gamma_{20} = \gamma_{21}$	$\gamma_{20} = \gamma_{21}$																			
11	$\gamma_{21} = \gamma_{22}$	$\gamma_{21}, \gamma_{22}, \gamma_{23}$	$\gamma_{21} = \gamma_{22}$	$\gamma_{21} = \gamma_{22}$																			
12	$\gamma_{22} = \gamma_{23}$	$\gamma_{22}, \gamma_{23}, \gamma_{24}$	$\gamma_{22} = \gamma_{23}$	$\gamma_{22} = \gamma_{23}$																			
13	$\gamma_{23} = \gamma_{24}$	$\gamma_{23}, \gamma_{24}, \gamma_{25}$	$\gamma_{23} = \gamma_{24}$	$\gamma_{23} = \gamma_{24}$																			
14	$\gamma_{24} = \gamma_{25}$	$\gamma_{24}, \gamma_{25}, \gamma_{26}$	$\gamma_{24} = \gamma_{25}$	$\gamma_{24} = \gamma_{25}$																			
15	$\gamma_{25} = \gamma_{26}$	$\gamma_{25}, \gamma_{26}, \gamma_{27}$	$\gamma_{25} = \gamma_{26}$	$\gamma_{25} = \gamma_{26}$																			
16	$\gamma_{26} = \gamma_{27}$	$\gamma_{26}, \gamma_{27}, \gamma_{28}$	$\gamma_{26} = \gamma_{27}$	$\gamma_{26} = \gamma_{27}$																			
17	$\gamma_{27} = \gamma_{28}$	$\gamma_{27}, \gamma_{28}, \gamma_{29}$	$\gamma_{27} = \gamma_{28}$	$\gamma_{27} = \gamma_{28}$																			
18	$\gamma_{28} = \gamma_{29}$	$\gamma_{28}, \gamma_{29}, \gamma_{30}$	$\gamma_{28} = \gamma_{29}$	$\gamma_{28} = \gamma_{29}$																			
19	$\gamma_{29} = \gamma_{30}$	$\gamma_{29}, \gamma_{30}, \gamma_{31}$	$\gamma_{29} = \gamma_{30}$	$\gamma_{29} = \gamma_{30}$																			
20	$\gamma_{30} = \gamma_{31}$	$\gamma_{30}, \gamma_{31}, \gamma_{32}$	$\gamma_{30} = \gamma_{31}$	$\gamma_{30} = \gamma_{31}$																			
21	$\gamma_{31} = \gamma_{32}$	$\gamma_{31}, \gamma_{32}, \gamma_{33}$	$\gamma_{31} = \gamma_{32}$	$\gamma_{31} = \gamma_{32}$																			
22	$\gamma_{32} = \gamma_{33}$	$\gamma_{32}, \gamma_{33}, \gamma_{34}$	$\gamma_{32} = \gamma_{33}$	$\gamma_{32} = \gamma_{33}$																			
23	$\gamma_{33} = \gamma_{34}$	$\gamma_{33}, \gamma_{34}, \gamma_{35}$	$\gamma_{33} = \gamma_{34}$	$\gamma_{33} = \gamma_{34}$																			
Here follows a repetition of Operations thirteen to twenty-three																							
24	$\gamma_{34} = \gamma_{35}$	$\gamma_{34}, \gamma_{35}, \gamma_{36}$	$\gamma_{34} = \gamma_{35}$	$\gamma_{34} = \gamma_{35}$																			
25	$\gamma_{35} = \gamma_{36}$	$\gamma_{35}, \gamma_{36}, \gamma_{37}$	$\gamma_{35} = \gamma_{36}$	$\gamma_{35} = \gamma_{36}$																			



As máquinas tinham sistema operacional?

Não.

A máquina diferencial e a máquina analítica não possuíam sistema operacional.

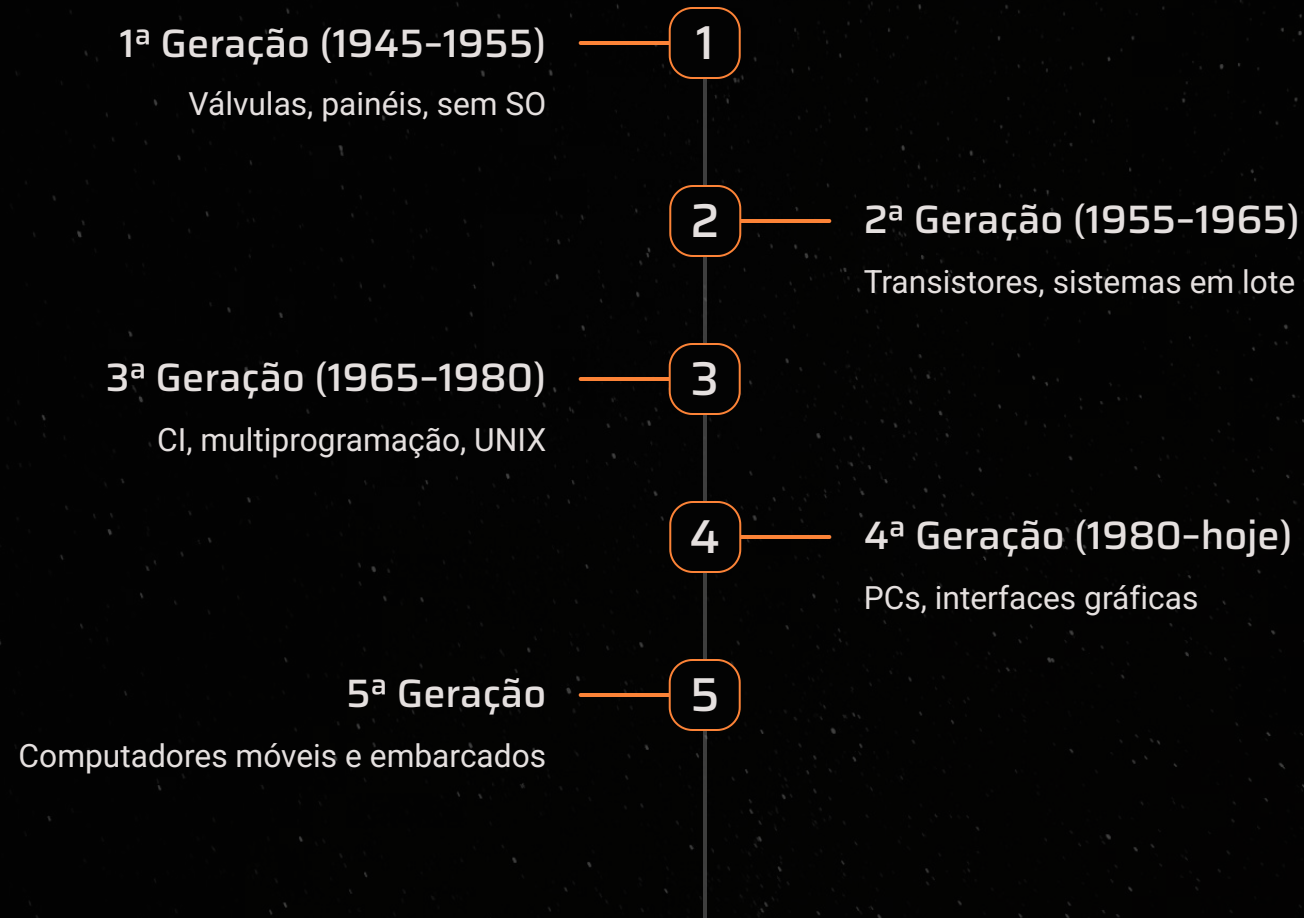
Mas já existiam elementos conceituais:

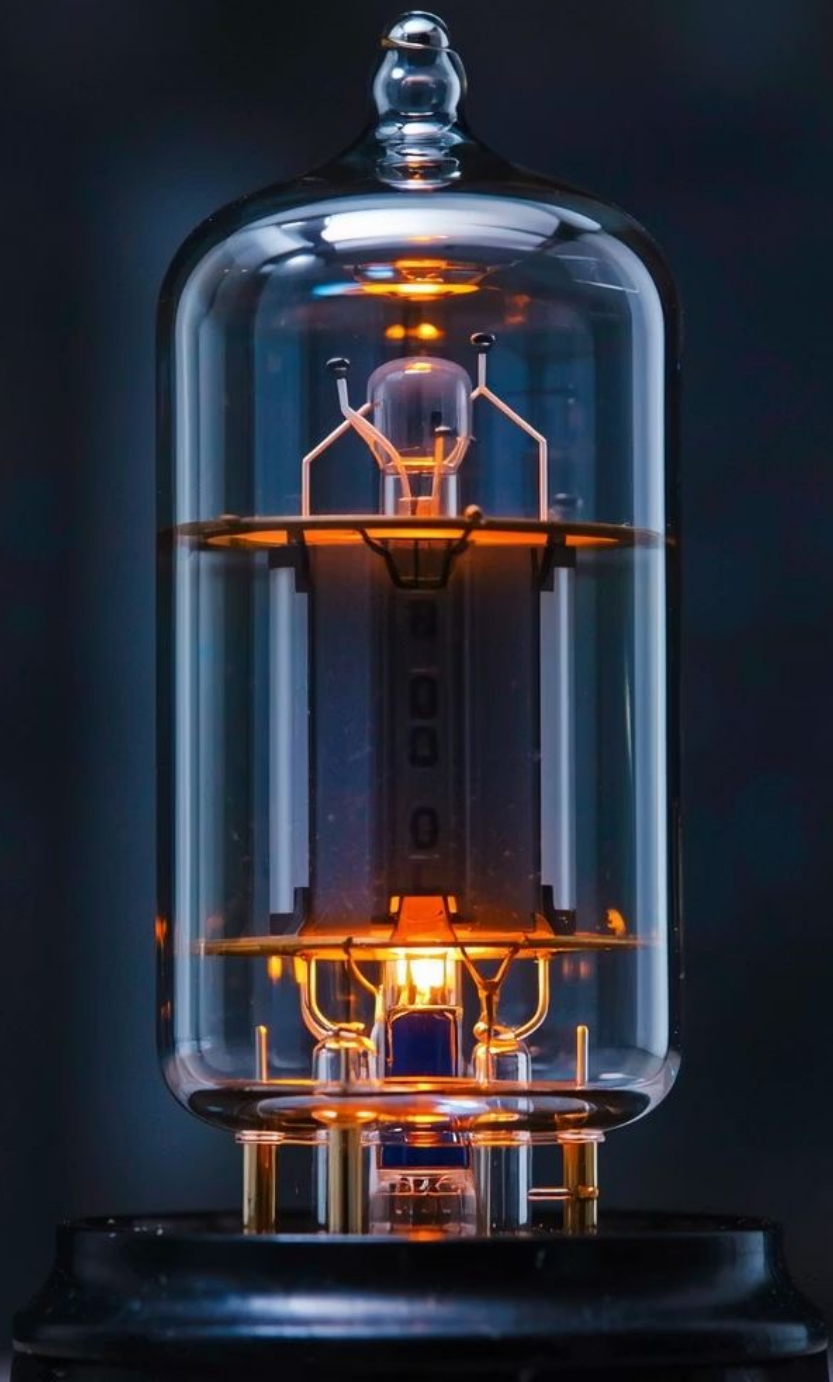
1. Máquina programável com instruções
2. Entrada e saída de dados
3. Processamento automático
4. Necessidade de organizar a execução

Esses elementos **anteciparam os problemas** que os sistemas operacionais viriam resolver.

A evolução por gerações

Tanenbaum organiza a evolução dos sistemas operacionais a partir das gerações de computadores. Cada geração traz novos desafios e novas soluções:





1ª GERAÇÃO – 1945 A 1955

Primeira geração: válvulas e painéis

Hardware

Máquinas baseadas em válvulas termiônicas — grandes, caras e pouco confiáveis

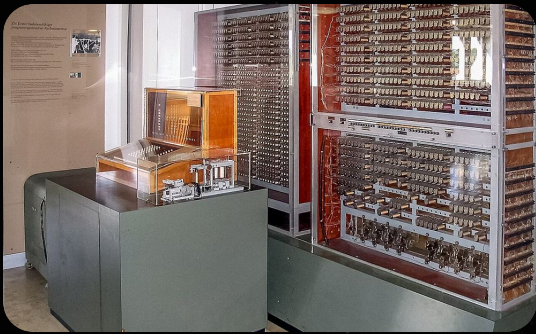
Programação

Linguagem de máquina e painéis de conectores físicos

Sistema operacional

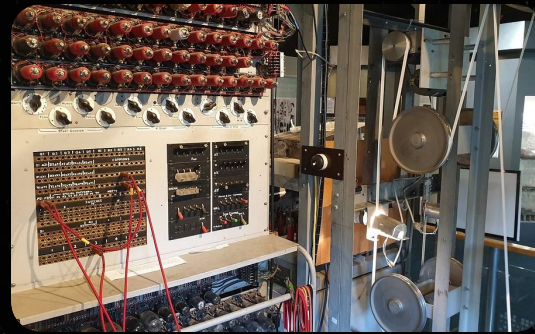
Praticamente inexistente — operação completamente manual

Máquinas pioneiras



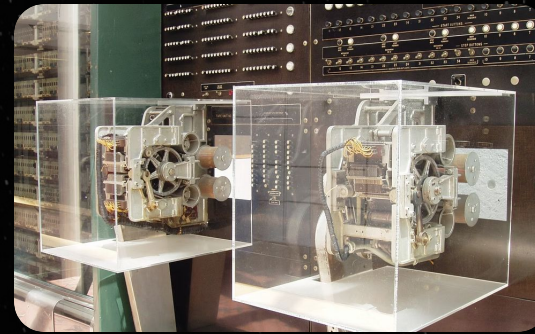
Z3 — Konrad Zuse

Primeiro computador eletromecânico programável, desenvolvido na Alemanha (1941).



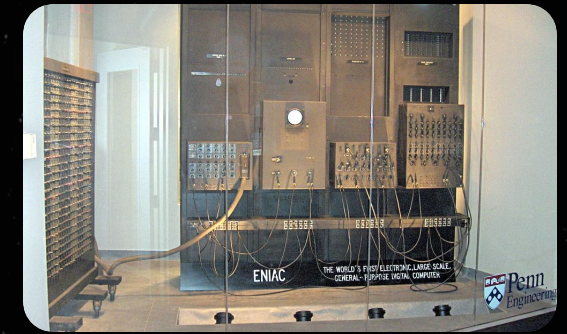
Colossus — Inglaterra

Usado para decifrar códigos nazistas durante a Segunda Guerra Mundial.



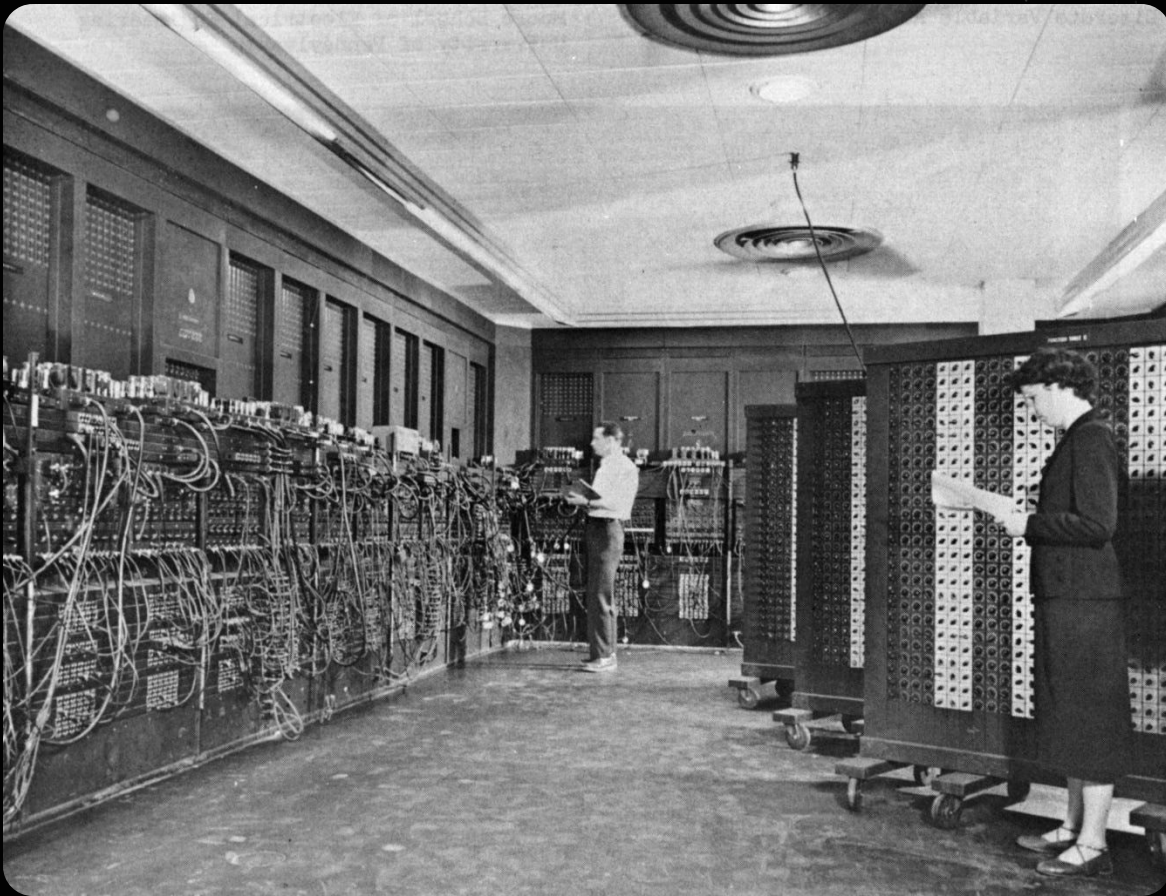
Mark I — Harvard

Computador eletromecânico desenvolvido em parceria com a IBM em 1944.



ENIAC — Penn

Um dos primeiros computadores eletrônicos de uso geral dos EUA.



ENIAC: computação eletrônica em grande escala

Desenvolvido durante a Segunda Guerra Mundial para **cálculos de trajetórias balísticas**.

- Ocupava uma sala inteira — 167 m²
- Cerca de 18.000 válvulas termiônicas
- Alto consumo de energia — 150 kW
- Programado por conexões físicas e chaves
- Falhas frequentes por queima de válvulas

Primeira geração

Quando programar era ligar fios

1

Planejar

Definir o algoritmo no papel

2

Configurar

Reconectar cabos e ajustar painéis fisicamente

3

Executar

Ligar a máquina e aguardar

4

Torcer

Esperar que nenhuma válvula queimasse no meio do processo

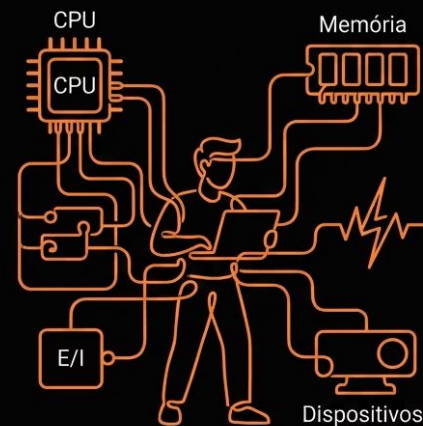
Programar hoje — escrever código em um editor e clicar em "executar" — é resultado de décadas de abstração construída sobre esse processo.

O problema da interação direta com o hardware

Sem sistema operacional, **todo o controle era responsabilidade do programador**:

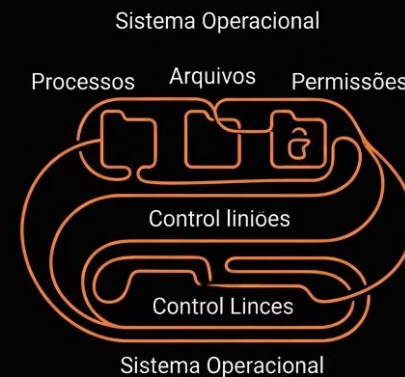
- Conhecimento profundo do hardware da máquina
- Preparação manual de cada tarefa
- Execução lenta e sujeita a falhas constantes
- Sem gerenciamento automático de recursos
- Sem proteção entre diferentes programas

❏ Esses problemas são exatamente o que os sistemas operacionais vieram resolver, geração após geração.



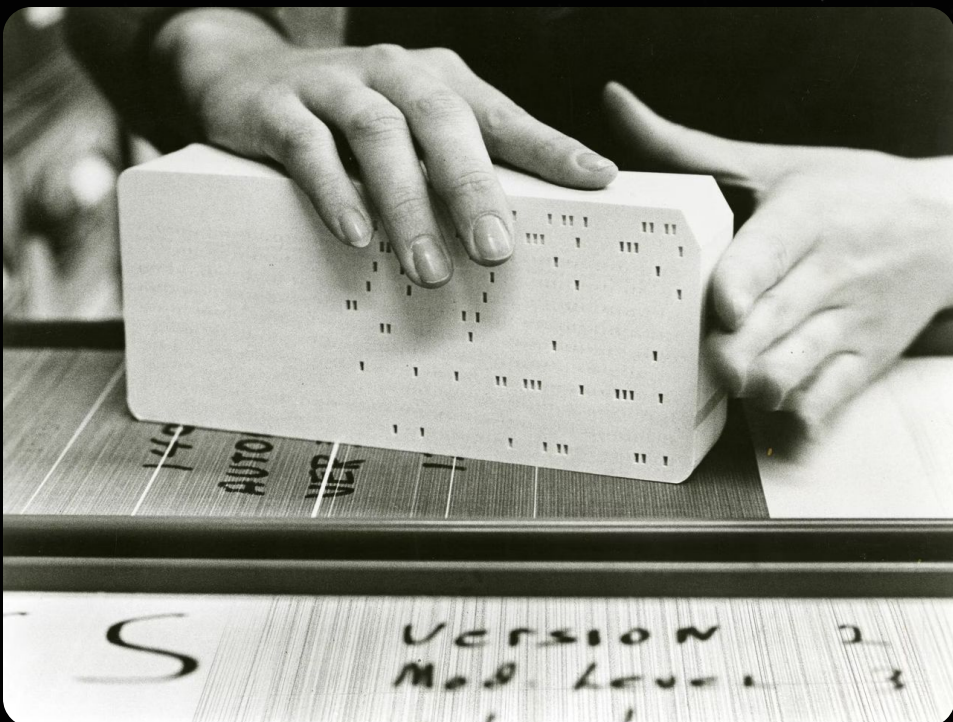
O programador controla tudo

Gerenciamento manual e complexo de recursos, dispositivos e erros.



Com SO

O SO abstrai recursos.
Gerenciamento simplificado e eficiente.



INÍCIO DOS ANOS 1950

A chegada dos cartões perfurados

Os cartões perfurados representaram um avanço significativo na forma de inserir programas nas máquinas:

- Programas podiam ser registrados e reutilizados
- Reduziu a necessidade de reconfiguração por cabos
- Entrada e saída tornaram-se mais organizadas
- Abriu caminho para processamento em lote

Ainda não havia sistemas operacionais completos, mas a direção estava traçada.

Os primeiros programas **monitores**

Antes dos sistemas operacionais completos, surgiram programas responsáveis por automatizar parte da operação da máquina. Esses programas eram chamados, em muitos contextos, de **monitores ou directors**.

Funções principais:

- carregar programas na memória;
- iniciar a execução de jobs;
- organizar a passagem de uma tarefa para outra;
- reduzir a dependência de intervenção manual do operador;
- apoiar o uso de cartões, fitas e impressoras.

1955

Director e Whirlwind I

- O **Director** foi desenvolvido no MIT para tornar o uso do Whirlwind I mais eficiente.
- Antes dele, a troca de tarefas exigia intervenção manual por cartões perfurados ou interruptores.
- O Director **usava fitas magnéticas** com instruções predefinidas.
- O Whirlwind I executava as tarefas automaticamente, uma após a outra, sem intervenção humana. Esse funcionamento antecipou a ideia de processamento em lote.
- Essa etapa marca a transição entre operação manual e controle automatizado da execução de programas.



Transição entre gerações

Da válvula ao transistor

A primeira geração provou que computadores podiam resolver problemas complexos — mas também expôs suas limitações críticas:

Baixa confiabilidade

Válvulas queimavam com frequência

Alto custo

Inacessível para a maioria das organizações

Grande porte

Ocupavam salas inteiras

Tempo ocioso

Desperdício enorme entre tarefas



2ª GERAÇÃO – 1955 A 1965

Segunda geração: transistores e sistemas em lote

Transistores

Substituem as válvulas – mais confiáveis, menores e mais baratos

Especialização

Surgem papéis distintos: projetista, operador, programador

Comercialização

Computadores chegam a empresas, governos e universidades

Sistemas em lote

Primeiras formas de organização operacional automatizada



Transistores e confiabilidade

↓90%

Tamanho reduzido

Em relação às válvulas termiônicas
equivalentes

↑10×

Confiabilidade

Vida útil muito maior que válvulas

↓95%

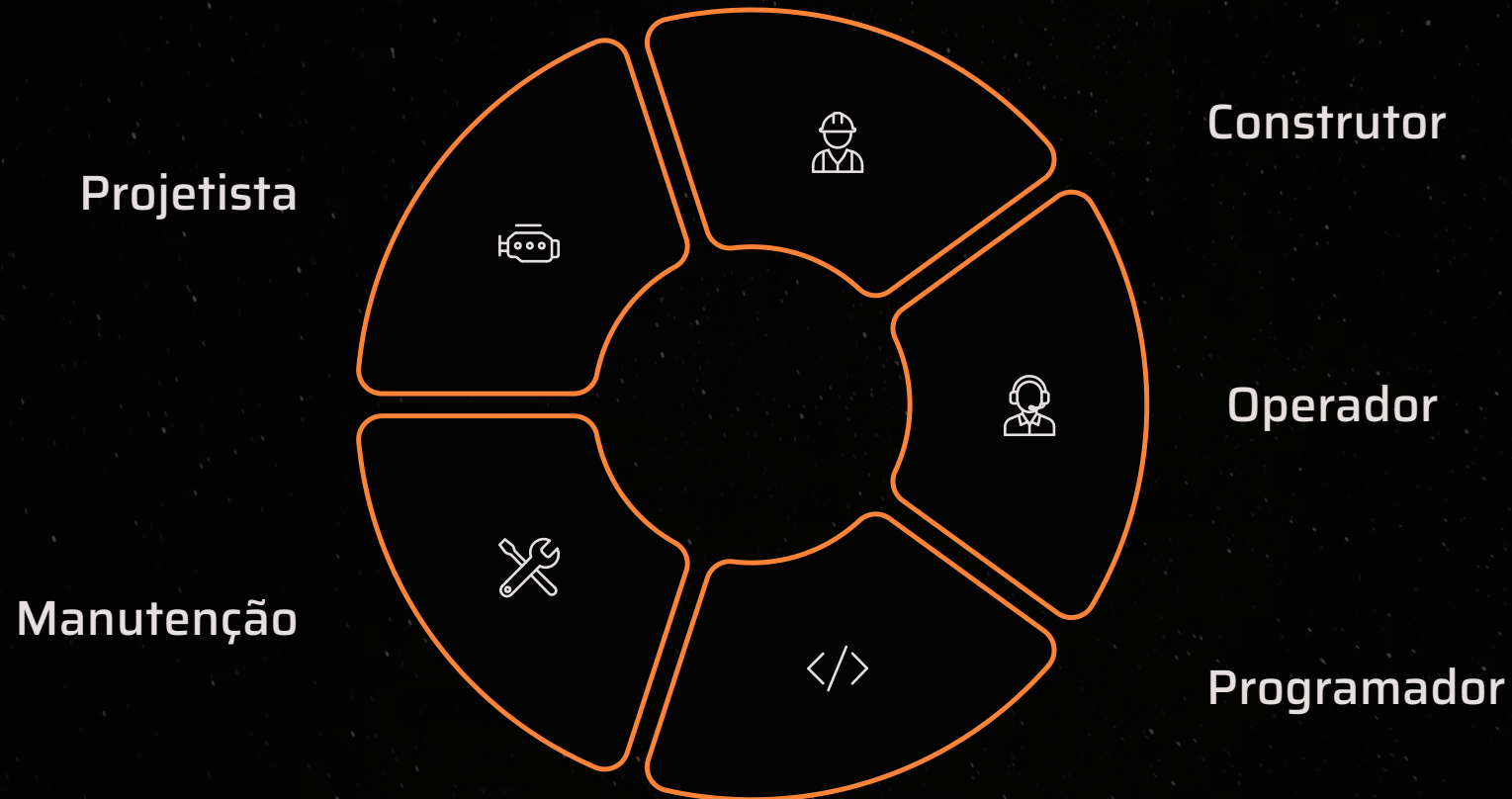
Consumo de energia

Computadores deixam de precisar de
salas climatizadas especiais

Esses ganhos tornaram possível a **comercialização em larga escala** dos computadores, abrindo o mercado para grandes empresas, governos e universidades nos anos 1950 e 1960.

Novos papéis na computação

Na segunda geração, a computação torna-se uma **atividade profissional e organizada**. A separação de papéis emerge naturalmente:



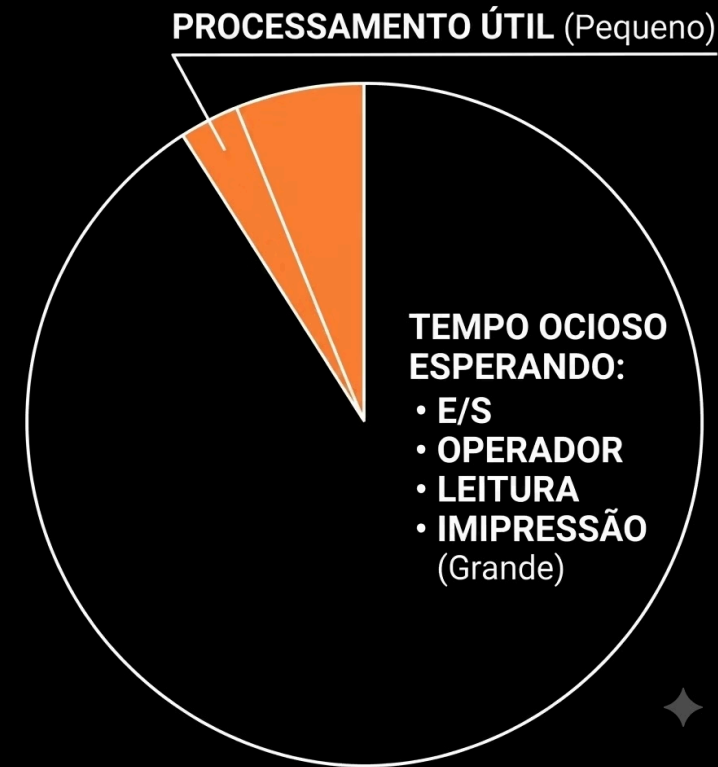
Computadores caros não podiam ficar parados

Como os computadores eram **extremamente caros**, o tempo ocioso representava prejuízo direto.

Situações que deixavam a CPU parada:

- Operador preparando manualmente a próxima tarefa
- Leitura lenta de cartões perfurados
- Espera por fitas magnéticas
- Impressão lenta dos resultados
- CPU aguardando operações de E/S

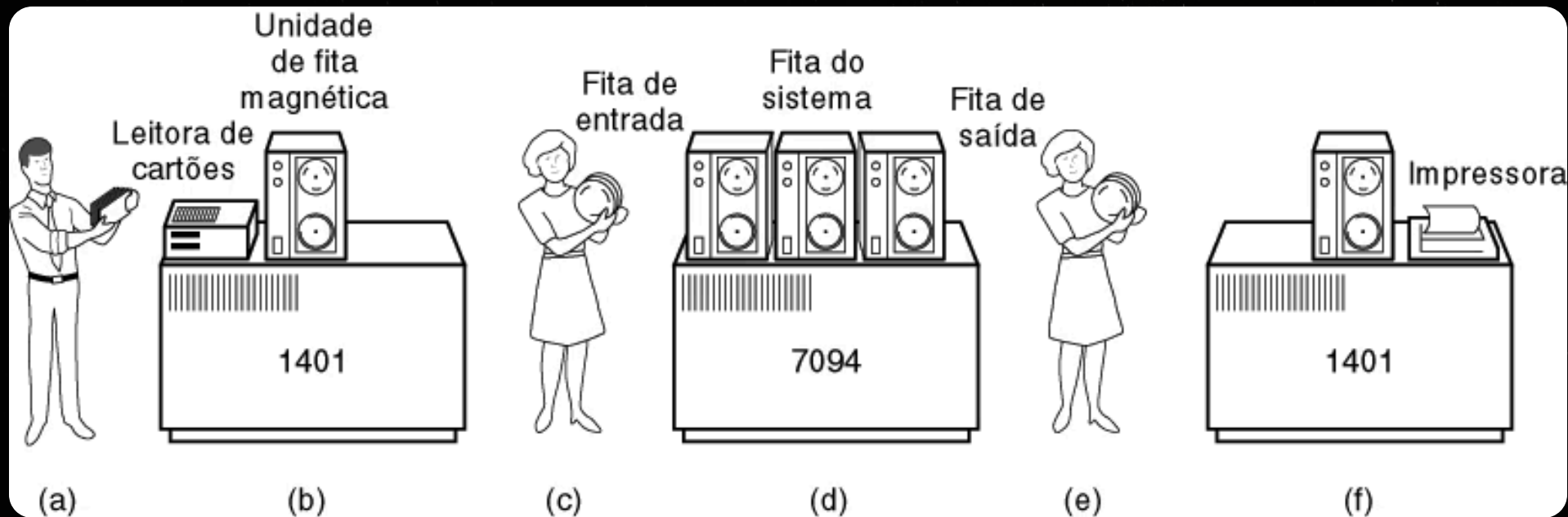
❏ Esse desperdício foi o principal motivador para o surgimento dos sistemas em lote.



Segunda geração

Sistemas em lote — *Batch*

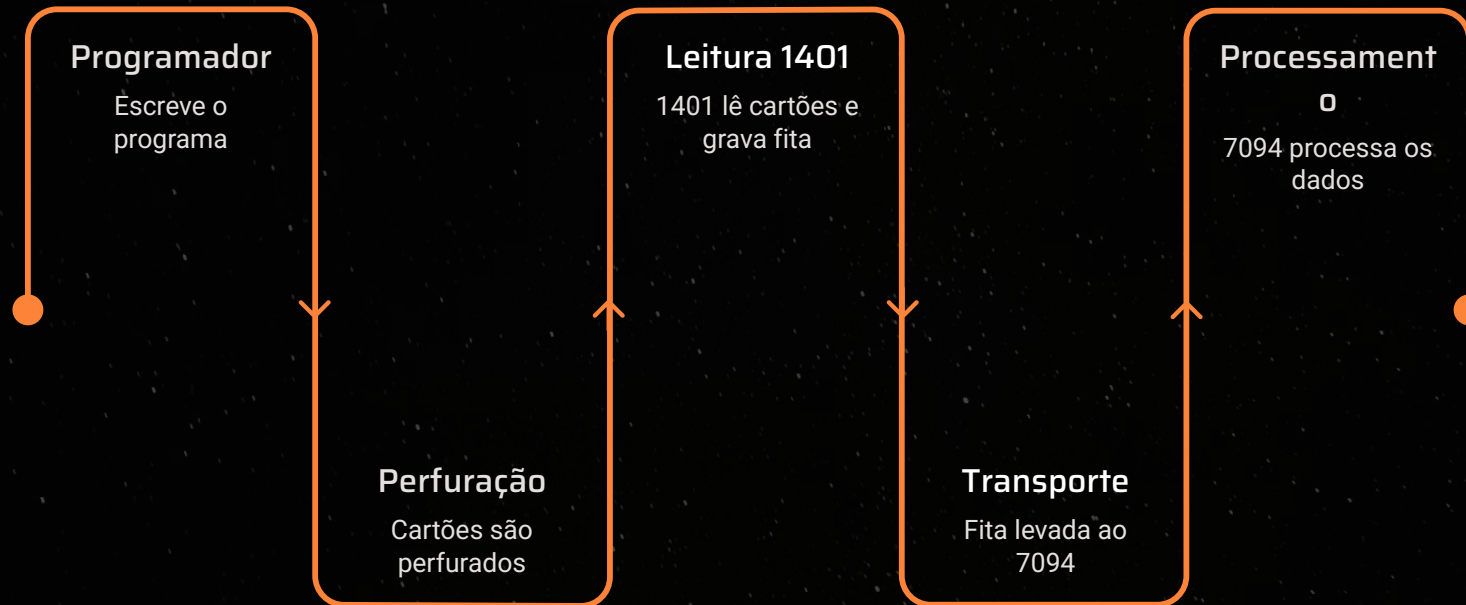
O processamento em lote se refere ao processamento de dados em grandes grupos ou lotes, em vez de em tempo real. É frequentemente usado no processamento de dados para aumentar a eficiência ao coletar tarefas e trabalhar nelas em conjunto mais tarde.



A ideia central era **reunir várias tarefas** e processá-las sequencialmente, sem interrupções manuais entre elas:

O fluxo de processamento em lote

O modelo clássico descrito por Tanenbaum envolvia **duas máquinas distintas** — uma para entrada/saída e outra para processamento:



Esse "passo duplo" era necessário porque as duas máquinas operavam em velocidades muito diferentes.

Já podemos falar em sistema operacional?

Ainda não plenamente...

Na segunda geração não havia SOs como os atuais, com gerenciamento completo de recursos e interface com o usuário.

Mas os primeiros elementos surgiram:

- Sistemas de fila de tarefas (*jobs*)
- Programas monitores de execução
- Controle automatizado entre tarefas
- Automatização parcial da execução

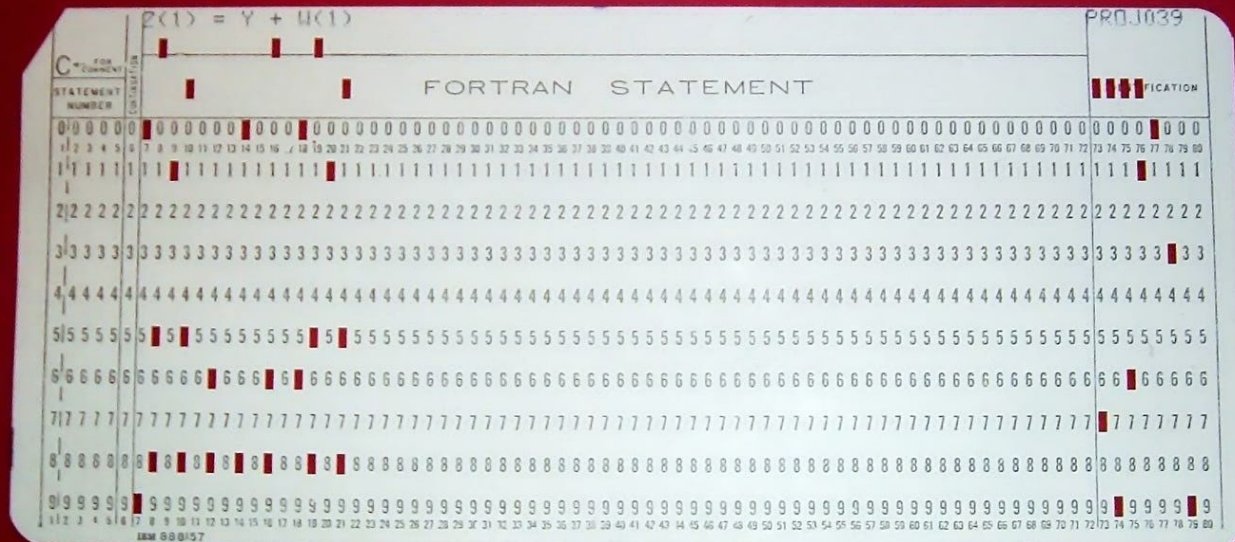
Exemplo histórico: **Fortran Monitor System (FMS)** – um dos primeiros monitores de lote.

SEGUNDA GERAÇÃO

FORTRAN

FORTRAN foi uma das **primeiras linguagens de alto nível** amplamente usadas, criada pela IBM entre 1954 e 1957.

- Foi a primeira linguagem de programação imperativa a **ser divulgada**
- Reduziu a dependência da linguagem de máquina
- O compilador era otimizado, pois os autores acreditavam que ninguém iria usar essa linguagem se seu desempenho não fosse comparável com a da linguagem assembly.



O que os sistemas em lote ainda não resolviam?

Sem interatividade

O usuário não interagía com o programa durante a execução — apenas submetia e aguardava.

Erros tardios

Falhas no programa só eram percebidas depois de horas — ou dias — de espera.

CPU ociosa em E/S

Durante operações de entrada e saída lentas, o processador ainda ficava parado.

Sem tempo real

Não havia como obter resposta imediata — o sistema não era interativo.

Esses problemas motivaram os grandes avanços da **terceira geração**.

3ª GERAÇÃO — 1965 A 1980

Terceira geração: circuitos integrados

A geração mais importante para o surgimento dos conceitos que usamos até hoje. Novas tecnologias, novos problemas, novas soluções.

Circuitos integrados

Mais poder, menos espaço e custo

Famílias de máquinas

Compatibilidade entre modelos —
IBM System/360

Multiprogramação

Vários programas na memória ao
mesmo tempo

Time-sharing

Múltiplos usuários simultâneos

Sistemas influentes

OS/360, CTSS, MULTICS, UNIX



1964

IBM System/360

Um projeto revolucionário: **unificar linhas completamente diferentes de computadores** em uma única família compatível.

- Diferentes modelos de desempenho, mesmo conjunto de instruções
- Atendia usos científicos e comerciais
- Software escrito para um modelo rodava nos demais
- Exigiu um sistema operacional que funcionasse em todos os modelos

Essa foi uma das apostas mais ousadas e lucrativas da história da IBM.

IBM System/360

OS/360 e a complexidade dos sistemas operacionais

O OS/360 se tornou um dos projetos de software mais complexos da história até então:

Escala sem precedentes

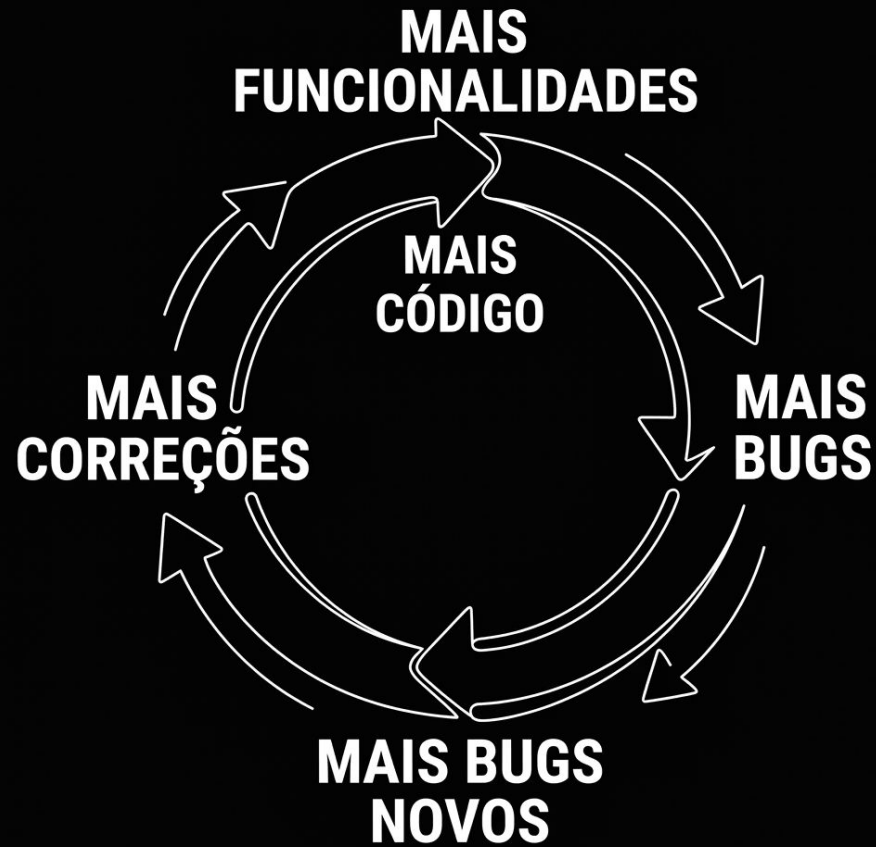
Milhões de linhas de código, centenas de programadores

Bugs em cascata

Cada correção tendia a introduzir novos problemas

Gestão impossível

Coordenar equipes grandes revelou limites da engenharia de software





Lições do OS/360

Fred Brooks, um dos líderes do projeto, documentou suas aprendizagens no livro *The Mythical Man-Month* (1975).



→ Lei de Brooks

Adicionar pessoas a um projeto atrasado o atrasa ainda mais.

→ Custo de comunicação

Em grandes equipes, coordenar é tão difícil quanto programar.

→ Bugs recorrentes

Corrigir um erro pode introduzir outro.

Conceito central da aula

Multiprogramação

O problema:

Quando um programa aguardava uma operação de **entrada ou saída** (leitura de disco, impressão), a CPU ficava completamente ociosa — um desperdício enorme.

A solução:

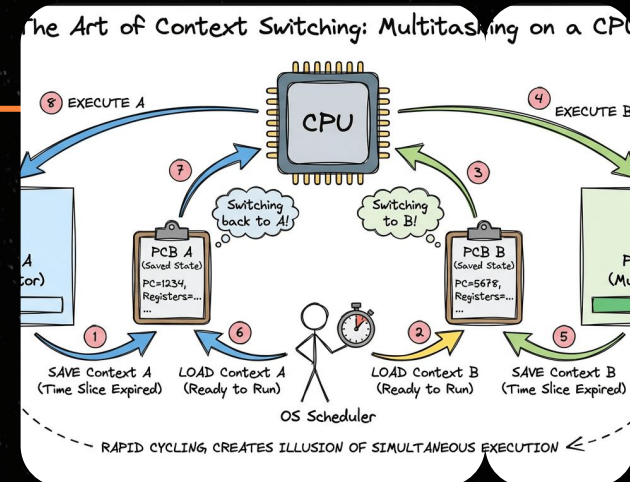
Manter **vários programas na memória** ao mesmo tempo. Quando um bloqueia, o SO executa outro.

O resultado:

- Melhor aproveitamento da CPU
- Redução drástica da ociosidade
- Base para os sistemas multitarefa modernos

Sem multiprogramação

CPU ociosa durante E/S, uma tarefa de cada vez

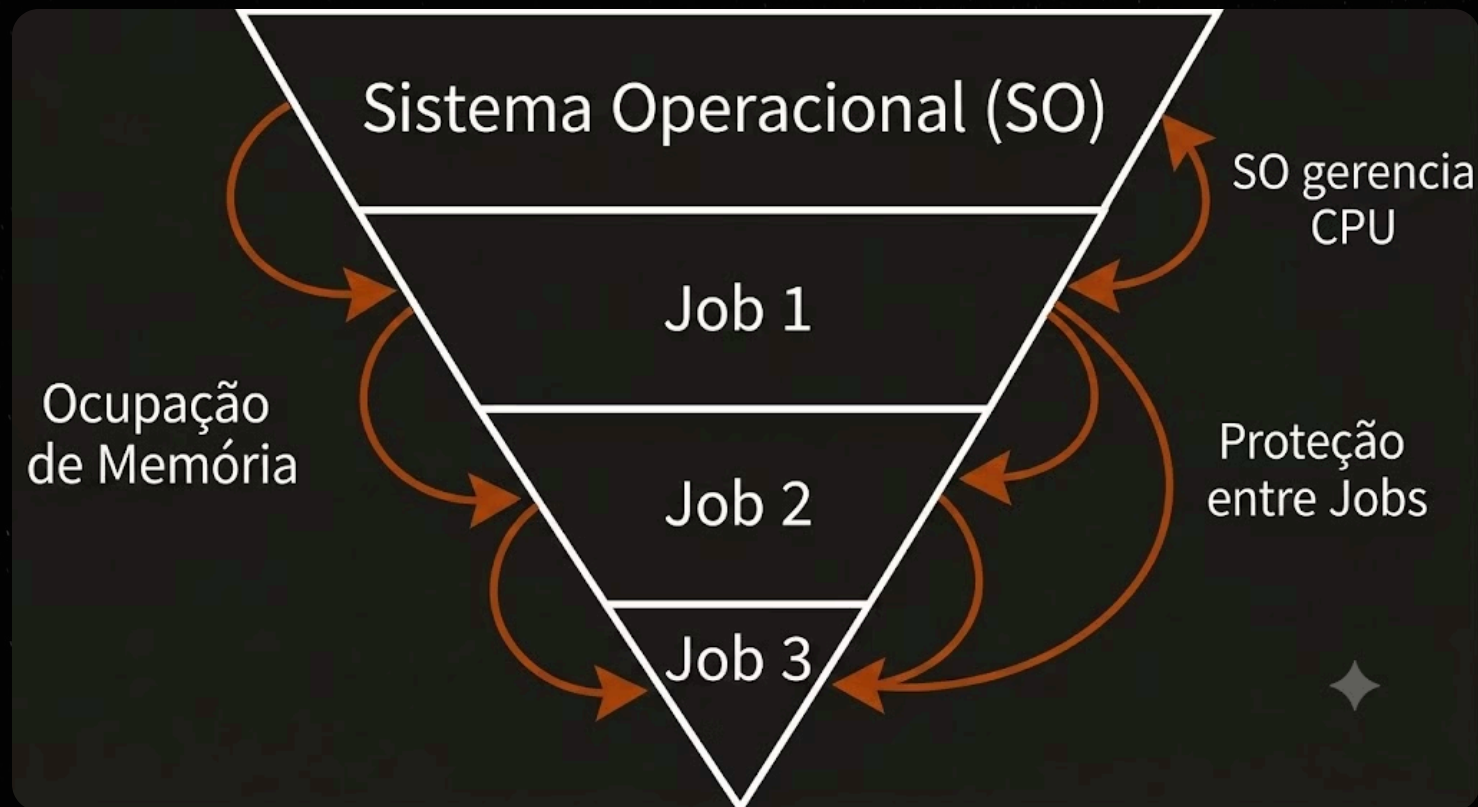


Com multiprogramação

CPU alterna entre tarefas, aproveitamento máximo

Vários programas na memória

Na multiprogramação, a memória principal é **dividida em partições**, cada uma alocada a um programa diferente:



Questões fundamentais que surgem: quem usa a CPU? Como proteger um programa do outro? Quem decide o próximo a executar? Essas questões formam o núcleo da disciplina de SOs.

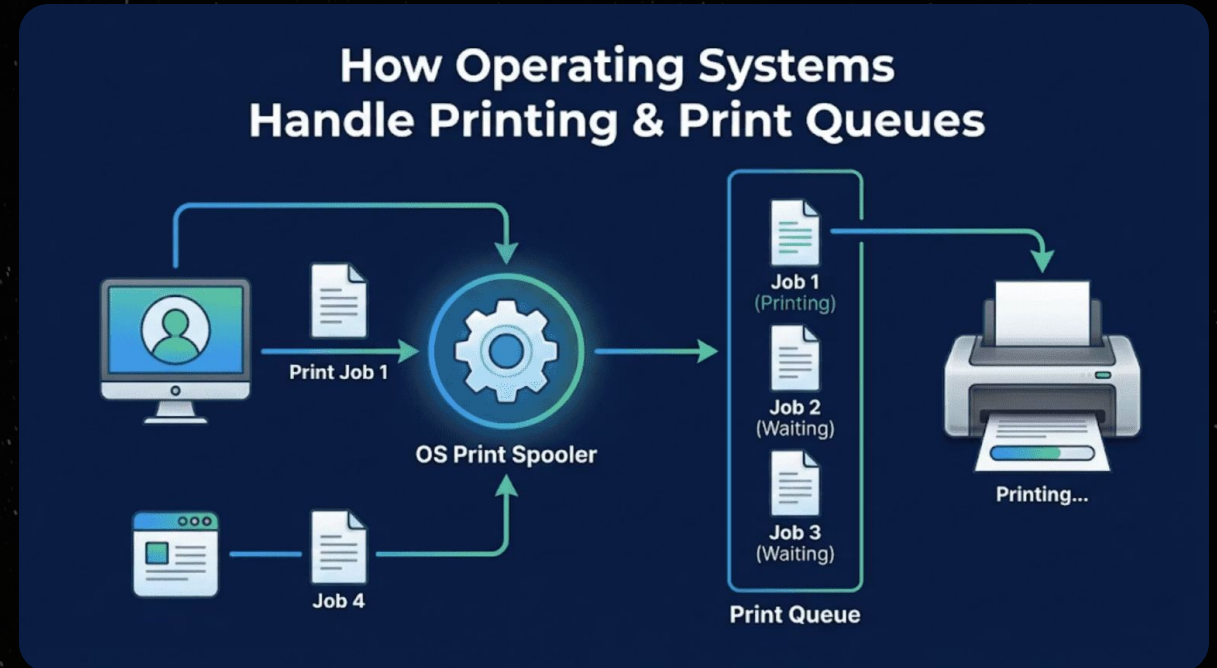
Spooling

Simultaneous Peripheral Operations On-Line — Operação Periférica Simultânea On-line.

Ideia fundamental: usar o disco como **área intermediária** entre a CPU e os periféricos lentos:

- Jobs lidos para o disco antecipadamente
- CPU processa sem esperar entrada direta
- Saída gravada em disco, enviada ao periférico depois
- Filas de entrada e saída organizadas automaticamente

📄 A fila de impressão e uma impressora usa exatamente esse princípio.



Compartilhamento de tempo (*Time-sharing*)

O **compartilhamento de tempo** permitiu que múltiplos usuários interagissem com o computador de forma simultânea e responsiva.

Terminal por usuário

Cada usuário conectado ao sistema por um terminal próprio

Fatias de tempo

O processador é dividido em pequenos intervalos — milissegundos

Pseudoparalelismo

Sensação de uso simultâneo, mesmo com uma única CPU

Interatividade

Usuário recebe resposta rápida — experiência próxima ao tempo real

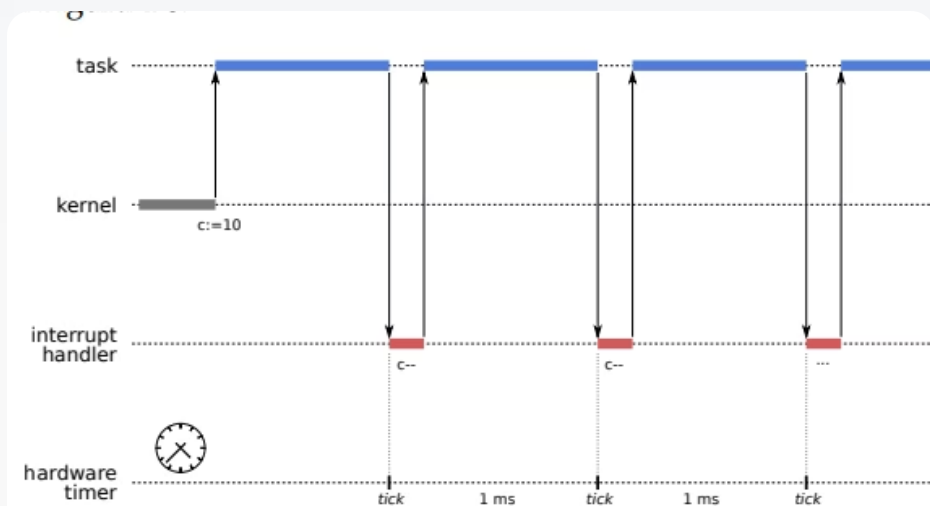


Figura 4.6: Dinâmica da preempção por tempo.

MIT — 1961

CTSS — Compatible Time-Sharing System

Um dos **primeiros sistemas de tempo compartilhado** bem-sucedidos da história.

- Desenvolvido no MIT sob liderança de John McCarthy e Fernando Corbató
- Influenciou diretamente o projeto MULTICS e, por extensão, o UNIX

O CTSS mostrou que **o computador poderia ser um recurso compartilhado**, não exclusivo.



MULTICS — Multiplexed Information and Computing Service

Um projeto **extremamente ambicioso**, desenvolvido pelo MIT, Bell Labs e General Electric — criticado em sua época, mas imensamente influente. Como descrito no artigo científico de 1965 "*Introduction and Overview of the Multics System*" por Corbató e Vyssotsky os principais objetivos do MULTICS eram

- Uso remoto e conveniente de terminais
- Uma grande variedade de configurações do sistema, alterados de forma transparente
- Um sistema de arquivos de alta confiança
- Suporte para informação compartilhada selecionada
- Estruturas hierárquicas de informação
- Suporte a vários tipos de aplicações e para vários ambientes de programação e interfaces para o usuário

MULTICS: avançado, mas complexo

- O MULTICS foi um sistema muito avançado para sua época
- Porém, era grande, complexo e dependia de máquinas *mainframe* caras.
- Suas ideias foram muito importantes, mas o projeto em si não se tornou o modelo dominante de mercado.
- O legado do MULTICS foi mais forte que sua adoção prática.

A criação do UNIX

O UNIX foi criado no fim da década de 1960, nos Bell Labs, por Ken Thompson e Dennis Ritchie.

Características importantes:

- sistema menor e mais simples que o MULTICS;
- inicialmente associado a minicomputadores;
- voltado ao uso por múltiplos usuários;
- forte relação com programação e ferramentas de desenvolvimento;
- grande influência na história dos sistemas operacionais.

Filosofia Unix

O Linux, criado por Linus Torvalds foi influenciado pela filosofia UNIX. É um sistema **UNIX-like**, ou seja, construído do zero, mas aderindo aos princípios e interfaces estabelecidos pelo UNIX.



Multiusuário e Multitarefa

Suporta múltiplos usuários interagindo com os recursos do computador simultaneamente, de forma independente e segura.



"Tudo é um arquivo"

A filosofia que trata dispositivos, pipes e sockets como arquivos simplifica a interação com o sistema operacional.



Sistema de Arquivos Hierárquico

Uma estrutura lógica e bem organizada (como `/bin`, `/etc`, `/home`).



Shell e Linha de Comando

A poderosa interface de linha de comando oferece controle granular e automação via scripts.



Utilitários Pequenos e Combináveis

Ferramentas focadas em uma única tarefa (`grep`, `sort`, `awk`) que podem ser encadeadas via pipes.



Compatibilidade POSIX

Aderência aos padrões POSIX garante portabilidade para aplicações desenvolvidas para sistemas UNIX.

Sistemas UNIX-like

1

Berkeley Software Distribution (BSD)

Desenvolvido na **Universidade da Califórnia, Berkeley**, a partir do código-fonte original do UNIX da AT&T.

Seu desenvolvimento principal começou em **1977**, marcando o início de uma linhagem de sistemas abertos.

2

Solaris

Criado pela **Sun Microsystems** no início dos anos **1990**, como um sucessor do SunOS.

Tornou-se um sistema UNIX comercial de alto desempenho, conhecido por sua robustez e escalabilidade em servidores.

3

FreeBSD

Lançado em **1993**, como um derivado do 386BSD, que por sua vez era baseado no 4.3BSD Net/2 de Berkeley.

É um dos sistemas operacionais de código aberto mais populares, amplamente utilizado em servidores, sistemas embarcados e desktops.

4ª GERAÇÃO — 1980 EM DIANTE

Computadores pessoais

A quarta geração colocou o computador nas mãos de qualquer pessoa — não apenas de grandes organizações. Sistemas como **MS-DOS**, **Mac OS** e **Windows** popularizaram o uso dos SOs, e a internet ampliou ainda mais essa transformação. Os conceitos desenvolvidos nas gerações anteriores — multiprogramação, spooling, time-sharing — foram herdados e refinados.

RY OF COMPU



1980



2000

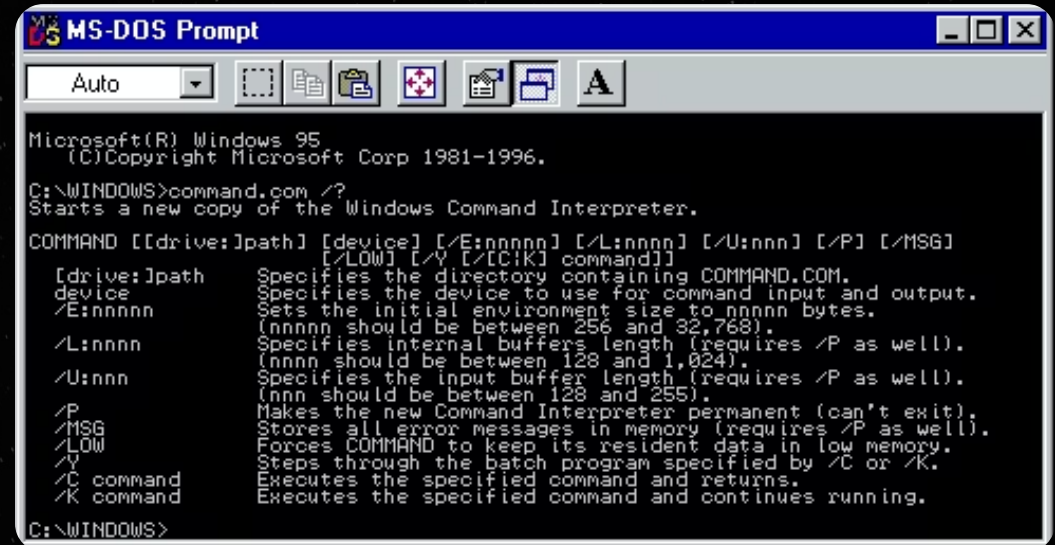
Quarta geração: computadores pessoais

- A partir da década de 1980, os chips com milhares de transistores favoreceram a era dos computadores pessoais.
- O preço dos computadores caiu da casa dos milhões para a casa dos milhares de dólares.
- Computadores passaram a chegar a usuários não especialistas.
- Isso criou a necessidade de sistemas mais simples, acessíveis e com interfaces amigáveis.

📌 Os desafios agora deixam de ser apenas técnicos e passa a envolver usabilidade e mercado.

MS-DOS: o sistema dos primeiros PCs

- Lançado pela Microsoft em 1981.
- Muito usado nos IBM PCs e computadores compatíveis.
- Interface baseada em comandos de texto.
- Sistema simples e limitado pelas características do hardware da época.
- Teve grande importância na popularização dos computadores pessoais.
- Em suas versões posteriores, incorporou funcionalidades presentes em sistemas como o UNIX.



```
MS-DOS Prompt

Auto

Microsoft(R) Windows 95
(C)Copyright Microsoft Corp 1981-1996.

C:\WINDOWS>command.com /?
Starts a new copy of the Windows Command Interpreter.

COMMAND [[drive:]path] [device] [/E:nnnnn] [/L:nnnn] [/U:nnn] [/P] [/MSG]
[/LOW] [/Y [/C|K] command]]

[drive:]path    Specifies the directory containing COMMAND.COM.
device          Specifies the device to use for command input and output.
/E:nnnnn        Sets the initial environment size to nnnnn bytes.
                 (nnnnn should be between 256 and 32,768).
/L:nnnn         Specifies internal buffers length (requires /P as well).
                 (nnnn should be between 128 and 1,024).
/U:nnn          Specifies the input buffer length (requires /P as well).
                 (nnn should be between 128 and 255).
/P             Makes the new Command Interpreter permanent (can't exit).
/MSG           Stores all error messages in memory (requires /P as well).
/LOW           Forces COMMAND to keep its resident data in low memory.
/Y            Steps through the batch program specified by /C or /K.
/C command     Executes the specified command and returns.
/K command     Executes the specified command and continues running.

C:\WINDOWS>
```

Windows

O Windows começou sua jornada como uma **interface gráfica de usuário** (GUI) rodando sobre o MS-DOS, com o objetivo principal de tornar o computador mais intuitivo e acessível para o usuário comum. O que antes era um ambiente de linha de comando complexo, transformou-se em uma experiência visual.



Windows 3.x

Interface gráfica inicial sobre o MS-DOS.



Windows 95/98/Me

Foco na usabilidade, plug-and-play e internet.



Windows NT/2000/XP

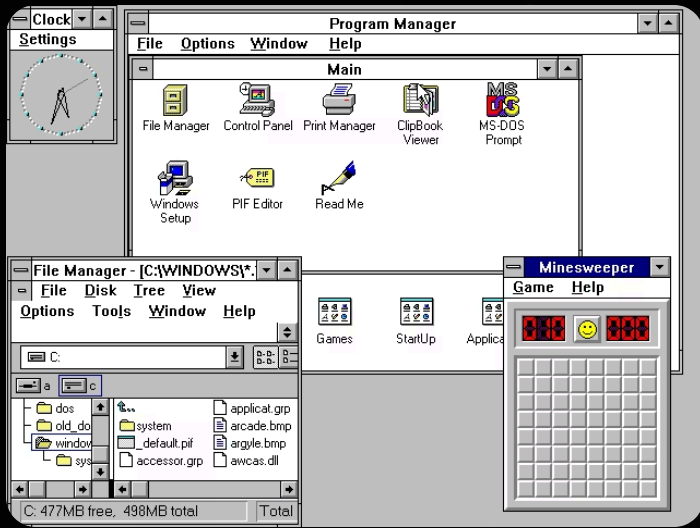
Arquitetura robusta, 32 bits, segurança e estabilidade.



Windows Moderno

Evolução contínua, integração e ecossistema.

A linha **Windows NT** foi um divisor de águas, estabelecendo uma base robusta com sua arquitetura de 32 bits, foco em segurança, multitarefa preemptiva e maior estabilidade, diferenciando-se das versões orientadas ao consumidor.



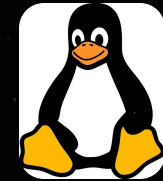
Windows 3.x



Windows XP

Do MINIX ao Linux

A partir do ideal de um sistema operacional didático, o **Linux** emergiu para se tornar um dos pilares da computação moderna, impulsionado por uma filosofia de código aberto e colaboração comunitária.



1987: MINIX

Andrew Tanenbaum desenvolve o MINIX, um sistema operacional didático inspirado na API do UNIX.

Desenvolvimento Comunitário

Adotado e desenvolvido rapidamente por uma comunidade global de programadores.

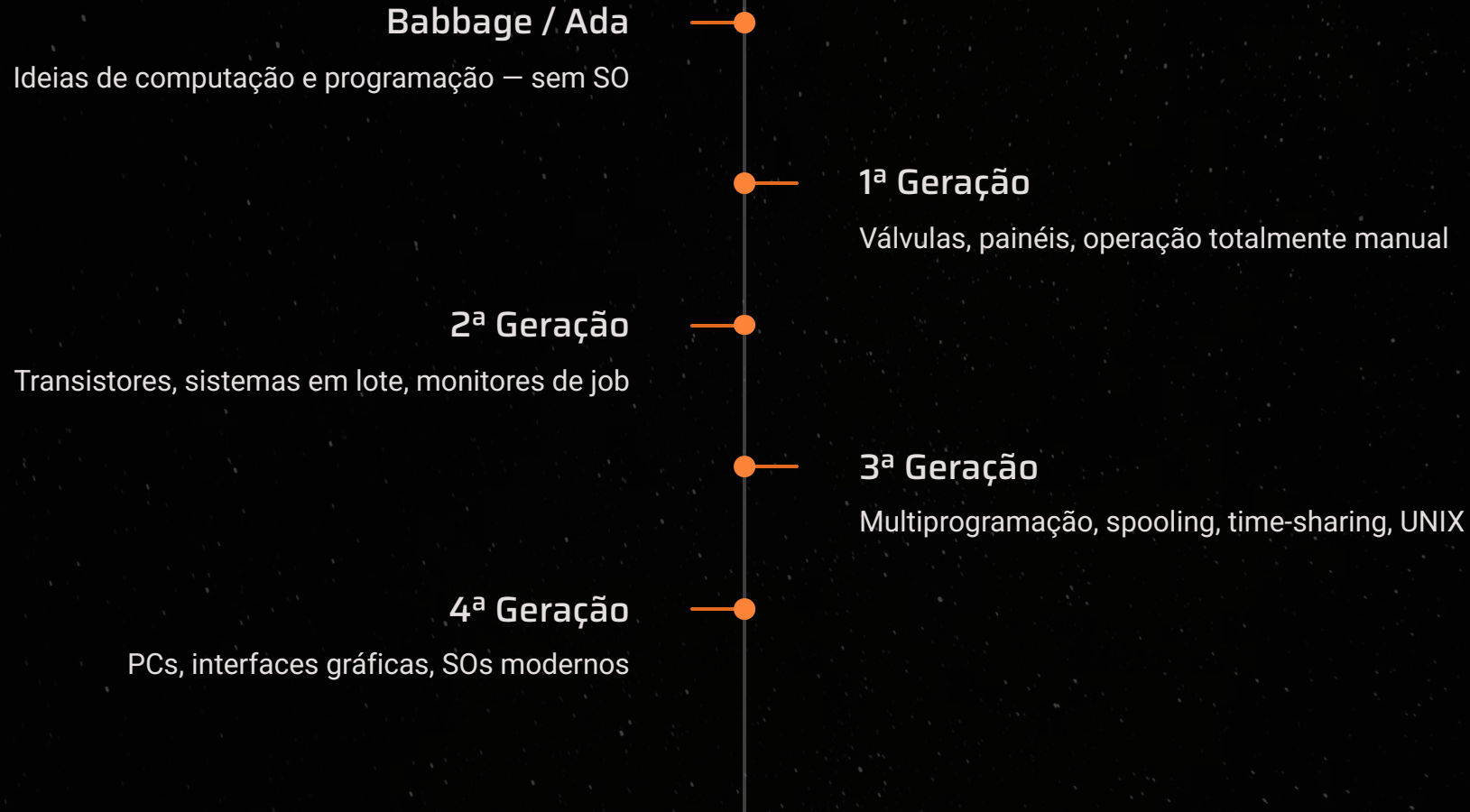
1991: Linus Torvalds

Linus Torvalds inicia o desenvolvimento do Linux, inicialmente como um núcleo de sistema operacional.

Ubiquidade

Tornou-se base para servidores, desktops, sistemas embarcados e Android, inspirando-se no UNIX, mas com código próprio.

Síntese da aula



📄 Muitos conceitos antigos permanecem nos sistemas atuais — reconhecê-los é a base para compreender os SOs modernos.

Dicas de filmes e documentários

- Revolution OS: <https://www.youtube.com/watch?v=errB71uSYLc>
- The code: story of Linux: <https://www.youtube.com/watch?v=XMm0HsmOTFI>
- Pirates of Silicon Valley

Referências bibliográficas

TANENBAUM, Andrew S.; BOS, Herbert.

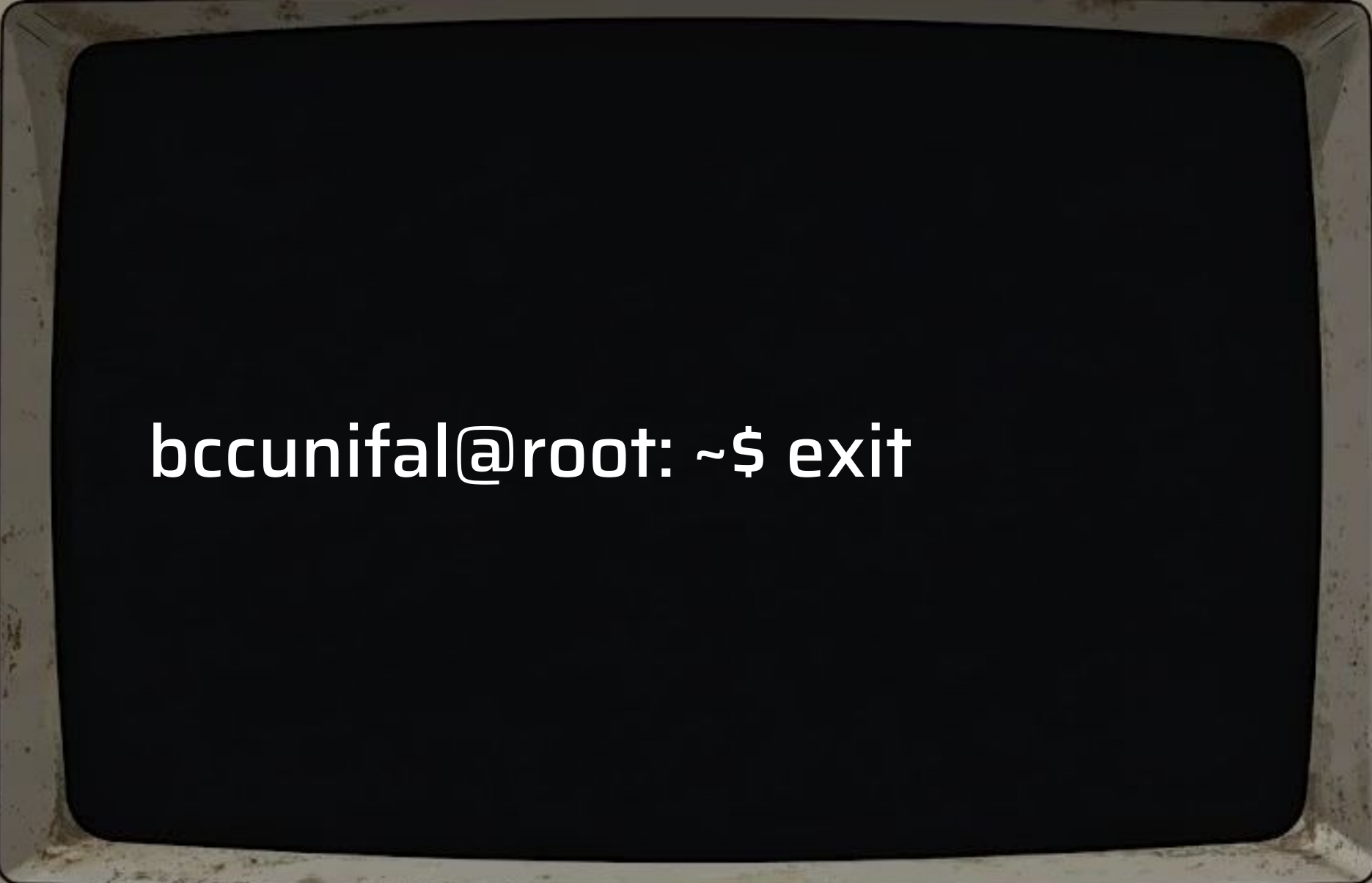
Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson, 2016.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg.

Fundamentos de Sistemas Operacionais. 9. ed. Rio de Janeiro: LTC, 2015.

MAZIERO, Carlos A.

Sistemas Operacionais: Conceitos e Mecanismos. Curitiba: UFPR, 2019.

A terminal window is displayed on a computer monitor. The monitor has a light-colored, slightly textured bezel. The terminal window itself has a black background with white text. The text shows a user prompt 'bccunifal@root:' followed by a tilde '~' and a dollar sign '\$', indicating the root user's home directory. The prompt is followed by the word 'exit', which has been entered but not yet executed. The overall scene is dimly lit, with some background elements like a keyboard and a mouse visible but out of focus.

```
bccunifal@root: ~$ exit
```